

Blockchain-based solution for a ***Music royalties system***

Chinmay Desai
D17C | 016
Batch A

1 | Problem and Solution

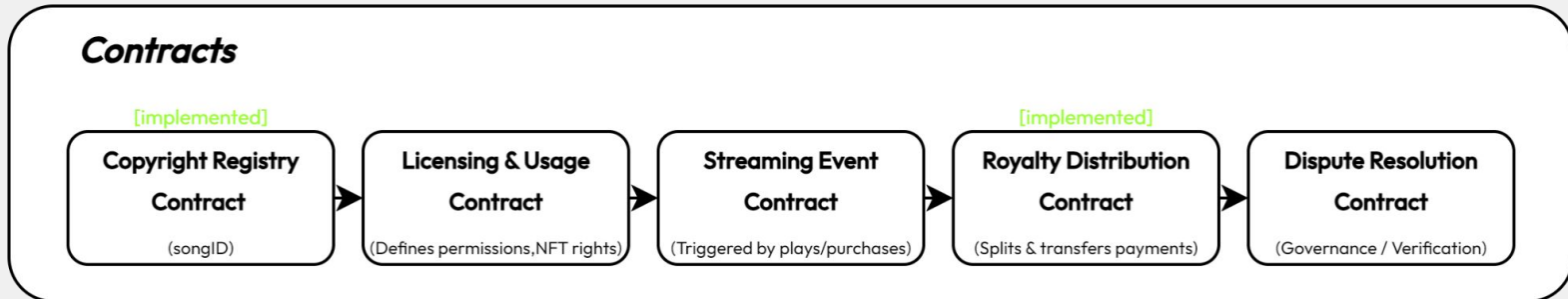
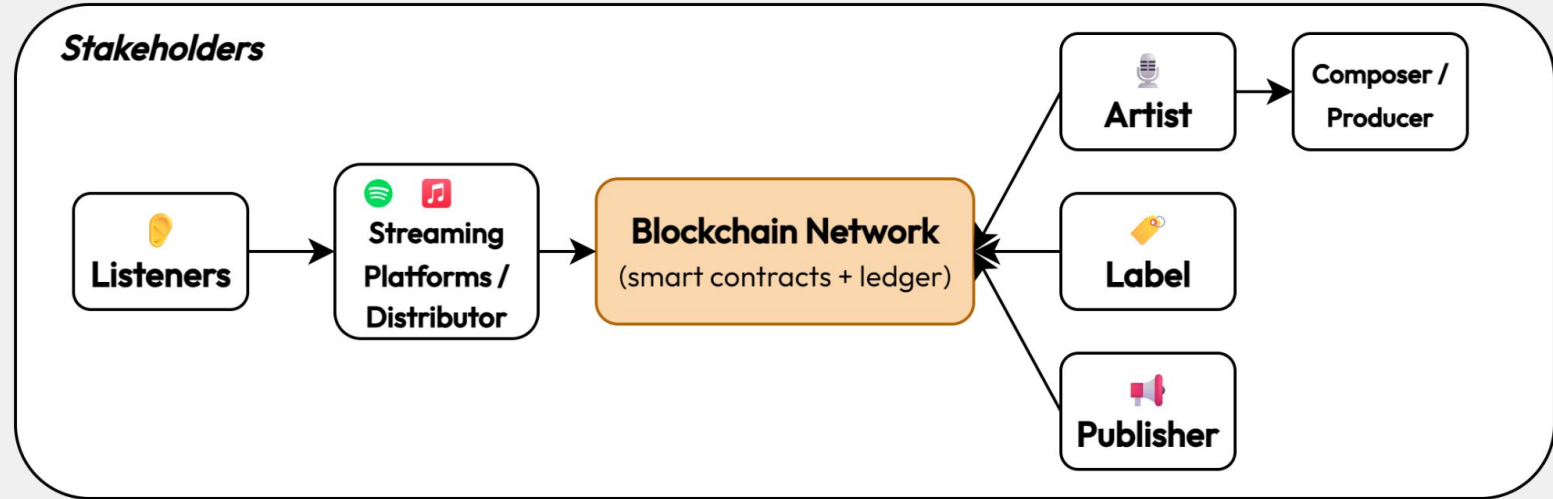
Underlying Problem:

- Inefficiency and Lack of Transparency
- Royalties take months or years to reach artists.
- Middlemen (labels, publishers, distributors) complicate tracking and payments.
- Opaque data — artists often can't verify how revenue is calculated.
- Disputes over ownership and revenue shares are common.

Possible Solution:

- Smart contracts automatically distribute royalties to all contributors in real time.
- Immutable ledger ensures transparent tracking of song ownership and usage.
- Tokenization allows micro-payments for every stream or download.
- Global accessibility enables fair compensation for independent and international artists.

2 | Stakeholders and Contracts



3.1 | Smart Contract 01 (RemixIDE)

✓ MUSICREGISTRY AT 0X089...FF

Balance: 0 ETH

REGISTERSONG

_title: "Comfortably Numb"

_collaborators: ["0x356344e8cff514f0e3a06e59E69"]

_shares: [100]

Calldata Parameters **transact**

getSong uint256 _songId

songCount

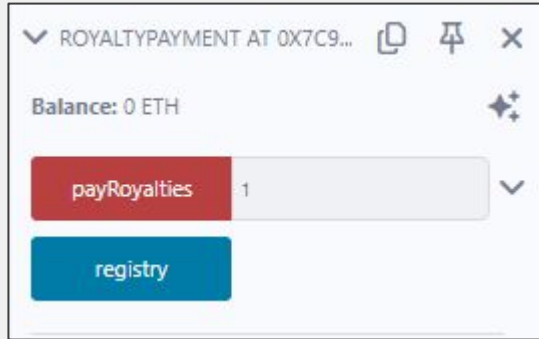
0: uint256: 1

songs uint256

musicregistry.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity >=0.7.0 <0.9.0;
3
4 contract MusicRegistry {
5     struct Song {
6         uint id;
7         string title;
8         address payable[] collaborators;
9         uint[] shares; // percentage shares, must add up to 100
10    }
11
12    mapping(uint => Song) public songs;
13    uint public songCount;
14
15    event SongRegistered(uint songId, string title);
16
17    function registerSong( undefined gas
18        string memory _title,
19        address payable[] memory _collaborators,
20        uint[] memory _shares
21    ) public {
22        require(_collaborators.length == _shares.length, "Mismatch between collaborators and shares");
23
24        uint totalShare;
25        for (uint i = 0; i < _shares.length; i++) {
26            totalShare += _shares[i];
27        }
28        require(totalShare == 100, "Shares must equal 100");
29
30        songCount++;
31        songs[songCount] = Song(songCount, _title, _collaborators, _shares);
32
33        emit SongRegistered(songCount, _title);
34    }
```

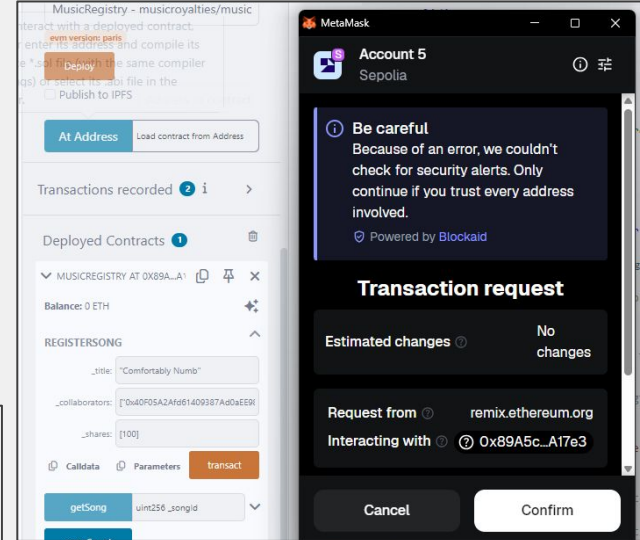
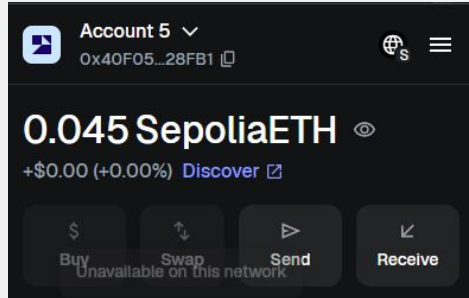
3.2 | Smart Contract 02 (RemixIDE)



royaltpayment.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity >=0.7.0 <0.9.0;
3
4 import "../musicregistry.sol";
5
6 contract RoyaltyPayment {
7     MusicRegistry public registry;
8
9     constructor(address _registry) {
10         registry = MusicRegistry(_registry);
11     }
12
13     event RoyaltiesPaid(uint songId, uint amount);
14
15     function payRoyalties(uint _songId) public payable {
16         require(msg.value > 0, "No payment sent");
17
18         (
19             , // title not needed here
20             address payable[] memory collaborators,
21             uint[] memory shares
22         ) = registry.getSong(_songId);
23
24         for (uint i = 0; i < collaborators.length; i++) {
25             uint payment = (msg.value * shares[i] / 100);
26             collaborators[i].transfer(payment);
27         }
28
29         emit RoyaltiesPaid(_songId, msg.value);
30     }
31 }
32
```

4 | Using Sepolia TestNet [Exp 07]



Transaction Hash:	0x6d8cb7ebc87806100cb7f101cd9f683e3848007cbde352f64cf64252c48ae233
Status:	Indexing This transaction has been included and will be reflected in a short while.
Block:	9412098
From:	0x40F05A2Afd61409387Ad0aEE981BbdD726628FB1
To:	0x89A5c5E63919BCf9358E52504f57A91af34A17e3
Value:	0 ETH
Gas Price:	1.500000011 Gwei (0.000000001500000011 ETH)

5 | Using Ganache [Exp 08]

← BACK		BLOCK 0	
GAS USED	GAS LIMIT	MINED ON	BLOCK HASH
0	6721975	2025-10-15 01:23:49	0x71568cff21ad6827b27695fc70ac7244fd920e07bc1a59bf76b7a7890277cb91

[illegible]

BLOCK 3	MINED ON 2025-10-15 01:30:28	GAS USED 616265	1 TRANSACTION
BLOCK 2	MINED ON 2025-10-15 01:28:48	GAS USED 185173	1 TRANSACTION
BLOCK 1	MINED ON 2025-10-15 01:27:43	GAS USED 1033816	1 TRANSACTION
BLOCK 0	MINED ON 2025-10-15 01:23:49	GAS USED 0	NO TRANSACTIONS

The screenshot displays the REMIX IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel is active, showing the deployment of a contract named 'MUSICREGISTRY - musicroyalties/music'. The 'Deploy' button is highlighted. Below the deployment options, the 'Transactions recorded' section shows a list of transactions, including 'Deployed Contracts' and 'MUSICREGISTRY AT 0X089_FF'. The 'Balance: 0 ETH' is displayed. The 'REGISTERSONG' function is shown with input fields for '_title: "Comfortably Numb"', '_collaborators: ["0x356344ebcfff140e3a06e59169"', and '_shares: [100]'. The 'transact' button is highlighted. On the right, the 'MetaMask' extension is open, showing the 'Account 6' (Localhost 7545) and a 'Transaction request' dialog. The dialog displays the request from 'remix.ethereum.org' interacting with '0x089ff...ff43a'. The network fee is '0.0037 ETH' and the speed is 'Speed'. The 'Confirm' button is highlighted.



Thank you!